

Turning construction data into decisions.

BuildSync AI is a construction data management platform for Indian real-estate developers: projects, tasks, materials, budgets, documents and daily site reports in one place, with an intelligence layer that reads them together and says what needs attention. This report sets out the fifteen points the system is built on — what each one does, how it is implemented, and what it is measured at.

PROJECT BuildSync AI	CONTEXT B.Tech Hackathon 2026 — Smart Construction Data Management	AUTHOR Vivek Ambariya
REPOSITORY main · d8c01d9	ENVIRONMENT Local development only	DATE 21 September 2026

4 ROLE WORKSPACES	13 COLLECTIONS	16 API ROUTERS	10k TRAINING ROWS
----------------------	-------------------	-------------------	----------------------

01 Problem and objectives

A construction project generates its truth in fragments. The programme lives in one spreadsheet, the bill of quantities in a supplier's PDF, the day's headcount in a WhatsApp message, the spend in an accounts package nobody on site can open. By the time those fragments are reconciled — typically at a monthly review — the decision they should have informed has already been made by default.

The cost of that lag is specific, not vague. A material with a three-week lead time runs out on a Tuesday; the purchase order needed to go out three weeks before anyone noticed the stock was low. A phase slips four percentage points a month for a quarter and the slip is only legible when it has become sixteen. Spend races ahead of the work it bought, and the cost performance index that would have shown it was never computed because nobody held both numbers at once.

What the system sets out to do

1. **Hold every record in one store** — projects, tasks, milestones, materials, expenses, documents and site reports, joined by reference rather than copied between systems.
2. **Compute every derived figure in exactly one place**, so the dashboard, the risk findings and the copilot can never quote different numbers for the same project.
3. **Rank what needs attention** rather than listing it — a sixteen-point schedule slip must outrank two overdue tasks, whichever was raised first.
4. **Give each role the surface its job needs**: a portfolio on a desk, a thumb-sized field app at a site gate, and an administration console — from one authorisation model.
5. **Make every number traceable** to the record it came from, and decline to show a number the data does not support rather than inventing one.

The governing constraint. Every figure the interface shows is computed in `backend/app/ai/engine.py`. Nothing recomputes a metric for itself — not the dashboard, not the reports module, not the copilot. That single rule is what makes the intelligence layer trustworthy rather than merely present, and it is the reason the architecture in point 03 has the shape it has.

02 What is in this report

01	Problem and objectives	p.1	02	What is in this report	p.1
03	System architecture	p.2	04	Technology stack	p.2
05	The four people	p.3	06	The site manager — a day on site	p.4
07	The contractor	p.6	08	The project manager	p.7
09	The admin	p.8	10	When the daily report is submitted	p.9
11	When a document is uploaded	p.10	12	When the system finds a risk	p.10
13	When someone asks BuildSync	p.11	14	Reports — who generates what	p.11
15	What it does not do yet	p.12			

03 System architecture

Six layers, each with one job, and a request that can only reach the database by passing through all of them in order.

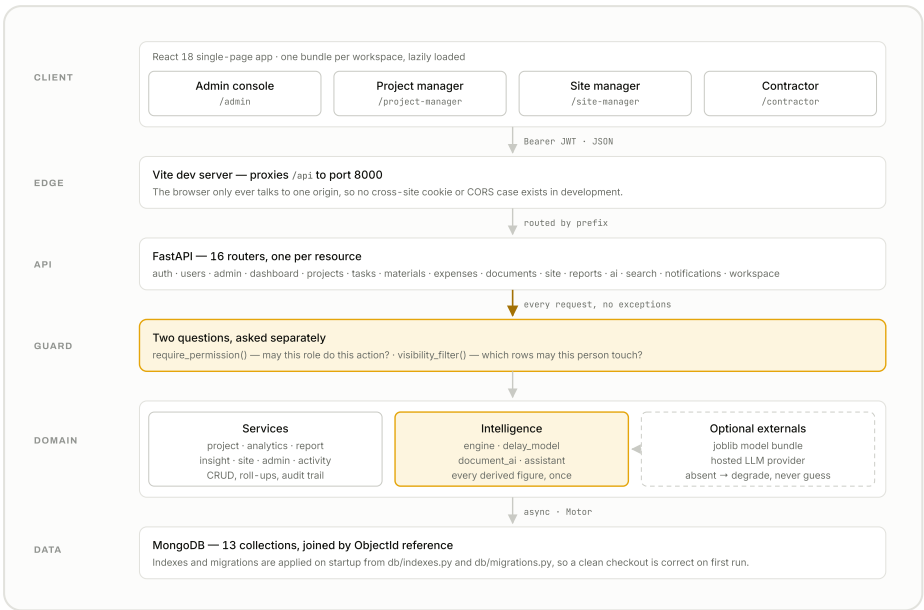


Figure 1 — Request path. The amber band is the only way through: a router never authorises for itself, and a service never reads a row the visibility filter has not already narrowed. Dashed edges are optional — the model bundle and the hosted LLM can both be absent, and the layer above them reports "unavailable" rather than substituting a value.

Why the layers are separated this way

`main.py` holds wiring only — lifespan, CORS, one error shape, router registration — so it stays readable as the surface grows. Authorisation is deliberately split into two questions that are answered in two different modules: *which actions* a role may perform lives in the permission matrix, and *which rows* a person may touch lives in the project service's visibility filter. An administrator passes both: every permission, and no filter. Because nothing else in the API decides authorisation for itself, adding a role or moving a capability is a change in one file.

04 Technology stack

Every choice here is load-bearing rather than decorative: the database is document-shaped because construction records are, the API is async because a dashboard request fans out into six collection reads, and the analysis layer is pandas because the work is genuinely tabular.

LAYER	CHOICE	WHY THIS ONE
Frontend	React 18, Vite 6, Tailwind 3.4, React Router 6	Route-level code splitting: a site engineer on a phone does not download the portfolio charts to record a headcount.
Visual	Recharts, GSAP + ScrollTrigger, Lenis, Lucide	Recharts for data marks, GSAP for the landing sequence, Lenis for scroll damping, one icon set throughout.
API	Python 3.11+, FastAPI 0.115, Pydantic 2.10	Schemas are the request and response contract, so validation and the OpenAPI docs at <code>/docs</code> are the same artefact.
Database	MongoDB with Motor 3.6 (async driver)	A project, its tasks and its extracted BOQ lines are naturally nested documents with a shifting shape.
Analysis	pandas 2.2, NumPy 2.2, scikit-learn 1.6	Group-bys and rolling rates over records; a linear regression for progress trend; a calibrated classifier for delay risk.
Security	PyJWT (HS256), bcrypt 4.2	Stateless twelve-hour tokens; passwords hashed with a per-password salt and never stored or logged in the clear.

The AI layer is pluggable rather than required: with no key configured, BuildSync runs a deterministic local engine — no network call, every number traceable to a record.

05 The four people, and what each is trusted with

BuildSync is not one product with four permission levels. It is four jobs, and each one gets the screen its job needs.

A director signing off a portfolio, a manager running three sites from a desk, an engineer standing at a gate with one hand on a phone, and a contractor who only wants to know what they are being paid to finish — these are not the same person with different checkboxes ticked. They work at different speeds, on different devices, and they need to be told different things. What they share is the record: when the engineer says a slab is poured, that is the same fact the director's progress curve moves on.

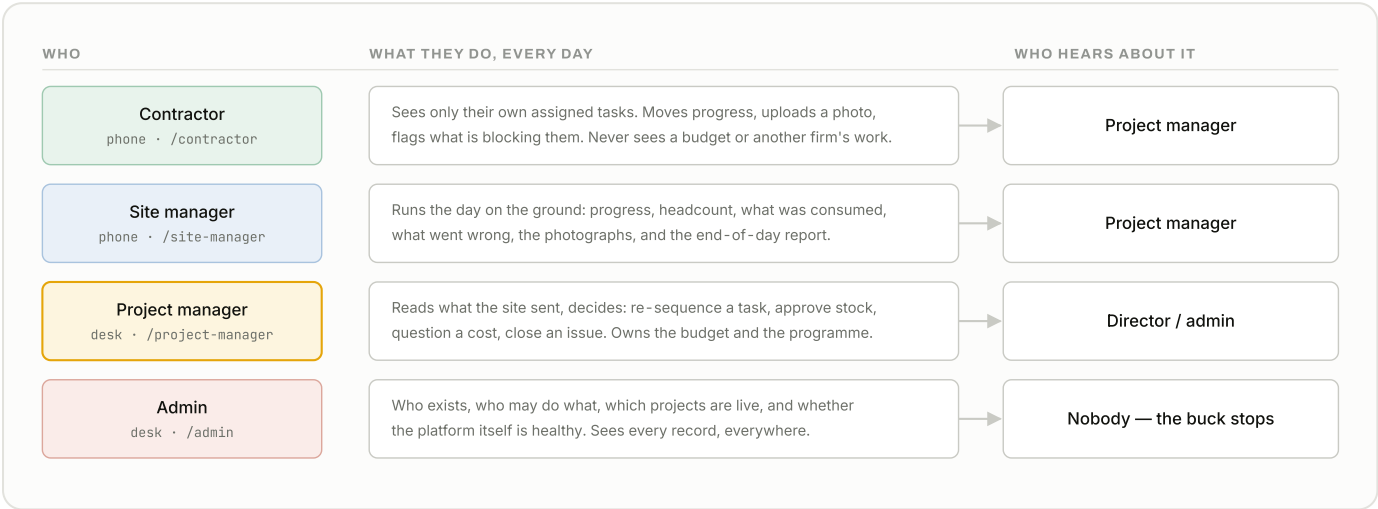


Figure 2 — Four jobs, one record. Work is reported upward and decisions come back down. Nothing in the chain is a copy: the contractor's progress entry is the same row the manager reads and the director's curve is drawn from.

PERSON	THEIR QUESTION, EVERY MORNING	WHAT THEY ARE DELIBERATELY NOT SHOWN
Contractor	What am I supposed to finish?	Budgets, expenses, other contractors' projects, the daily report
Site manager	What does the crew do today?	Every figure describing money — budget, spend, unit cost, invoice value
Project manager	What is going wrong, and what do I do about it?	Creating accounts, changing roles, removing a project from the portfolio
Admin	Is the organisation and the platform in order?	Nothing

The last column is enforced on the server, not in the interface. The site app's replies are stripped of money before they are sent, so a curious person with the browser console open still cannot read a budget.

06 The site manager — a day on site

The busiest user of the system, and the one whose screen was designed last, on purpose — because everything the others read is something this person typed in with one thumb.

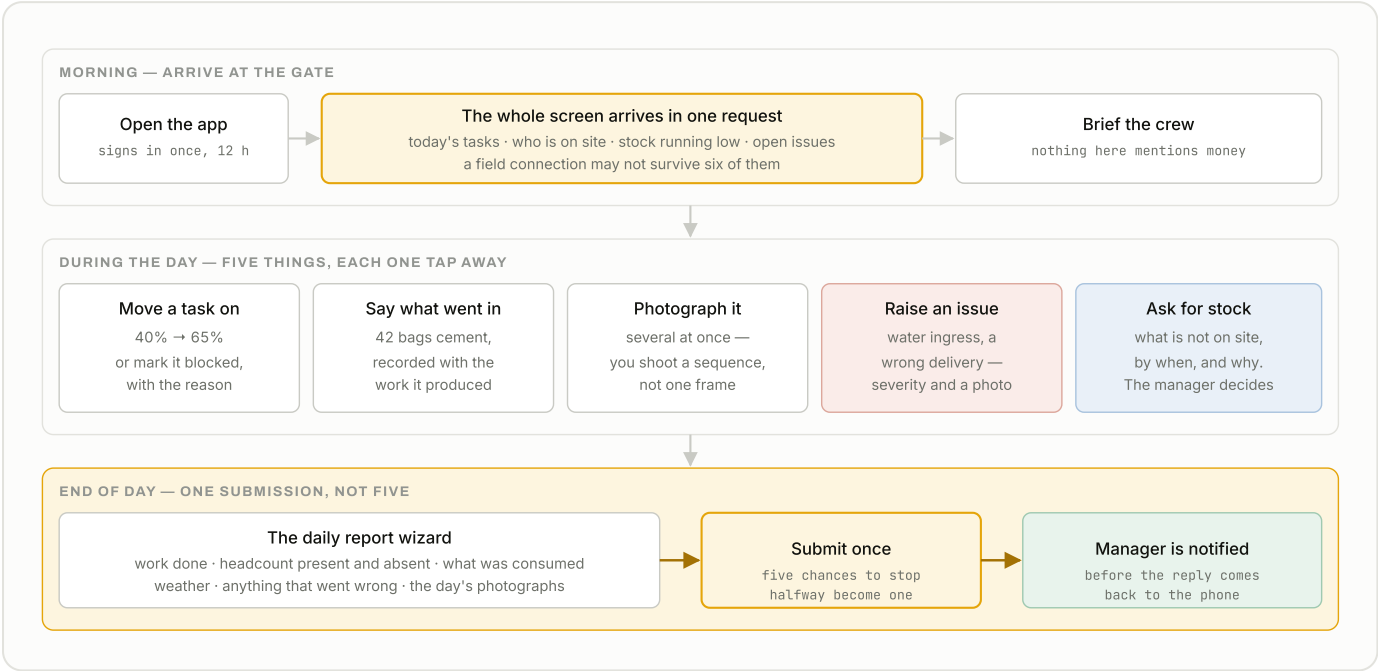


Figure 3 — The site manager's day. The shape of the day drove the shape of the screen: one load in the morning because the connection is bad, five one-tap actions during it because both hands are rarely free, and one submission at the end because the day is remembered as a whole, not as five separate forms.

The morning screen

Opening the app loads one screen and makes one request. It answers four questions at once: what is due today or already underway, how many people are on site, which materials are close to running out, and what is still open and unresolved. A task closed three weeks ago is not today's work; one closed this morning is, so the person can see what they have already got through.

The five things they do during the day

1. **Move a task on.** Progress and status are kept honest with each other — putting a task at 100% closes it, and moving a task off zero starts it, without anyone having to remember to change a dropdown as well. What they *cannot* do is move a deadline or reassign the work to someone else; that is the manager's decision, and the narrower screen is deliberate.
2. **Record what was consumed** — as part of the progress entry, not on a separate stock screen. Forty-two bags of cement went into the pour that took the slab from 40% to 65%; those are one event, so the system asks once and draws the stock down itself.
3. **Photograph the work**, several frames at a time, and the photographs attach themselves to the progress entry or the report they belong to.
4. **Raise an issue** when something is holding the work up, with a severity and a photograph. The project manager is notified before the screen returns.
5. **Request material** the site does not have, saying how much and by when. It arrives with the manager as a decision to make, not a message to read.

Why the site app is a separate application. The portfolio answers *how is the business doing*. Everything on this surface answers *what do I need to do today*, for one person, on one site, on a phone. They are different questions, so they are different applications sharing one set of records — which is also why the field app is a separate download: an engineer recording a headcount at a gate should not be waiting on the portfolio's charts.

What the screen is built for

- **Nothing tappable is under 52 pixels.** That is a gloved thumb, not a mouse pointer, and it is why the field surface looks heavier than the office one despite using the same palette.
- **Controls sit where a thumb already is**, toward the bottom of the screen, rather than where a cursor would go.
- **Higher contrast and heavier weight**, because the screen is being read outdoors in daylight.
- **No money, anywhere.** Budget, spend, unit cost and invoice value are removed from every reply sent to this surface. A site manager reports what was built and what it consumed; what it cost is not their screen, and the server does not send it.

07 The contractor — their own work, and nothing else

The same field application as the site manager, with two doors locked and the portfolio shrunk to the projects they were actually given.

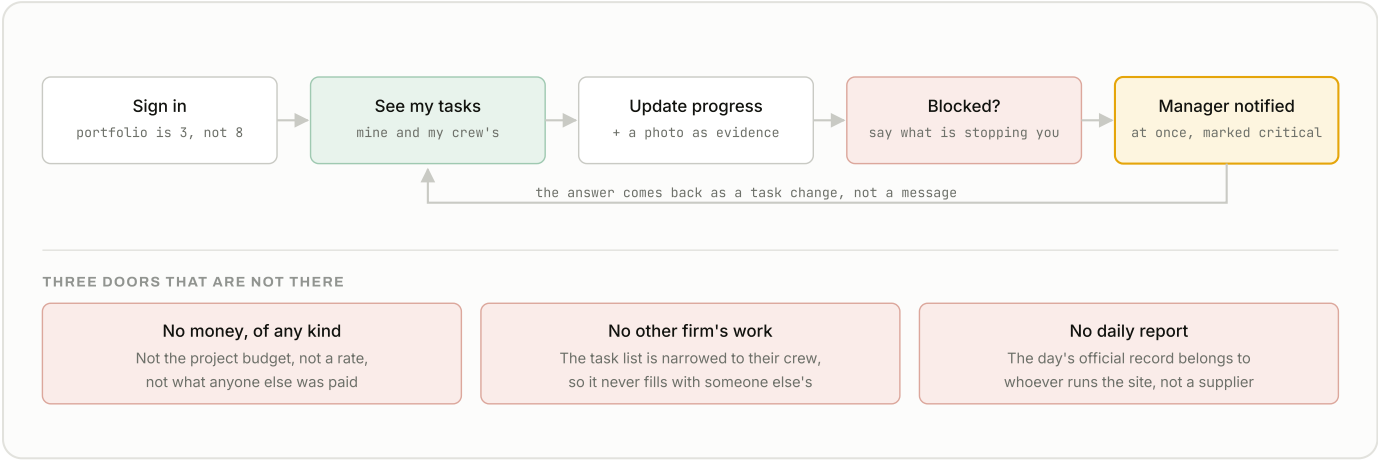


Figure 4 — The contractor's loop. Two routes that exist for the site manager — today's work and the daily report — are simply not mounted for a contractor, so there is no screen to find rather than a screen that refuses. The three red panels are enforced on the server as well, so the absence is real and not cosmetic.

The effect is easiest to see by signing in as one. The portfolio drops from eight projects to three, the task list to their own crew's work, and the write actions disappear from the interface — while the API refuses the same calls independently, so a hand-crafted request gains nothing.

08 The project manager — deciding, not collecting

Everything the site sent yesterday is already in. The job is no longer to chase it — it is to work out which three things out of forty actually matter today.

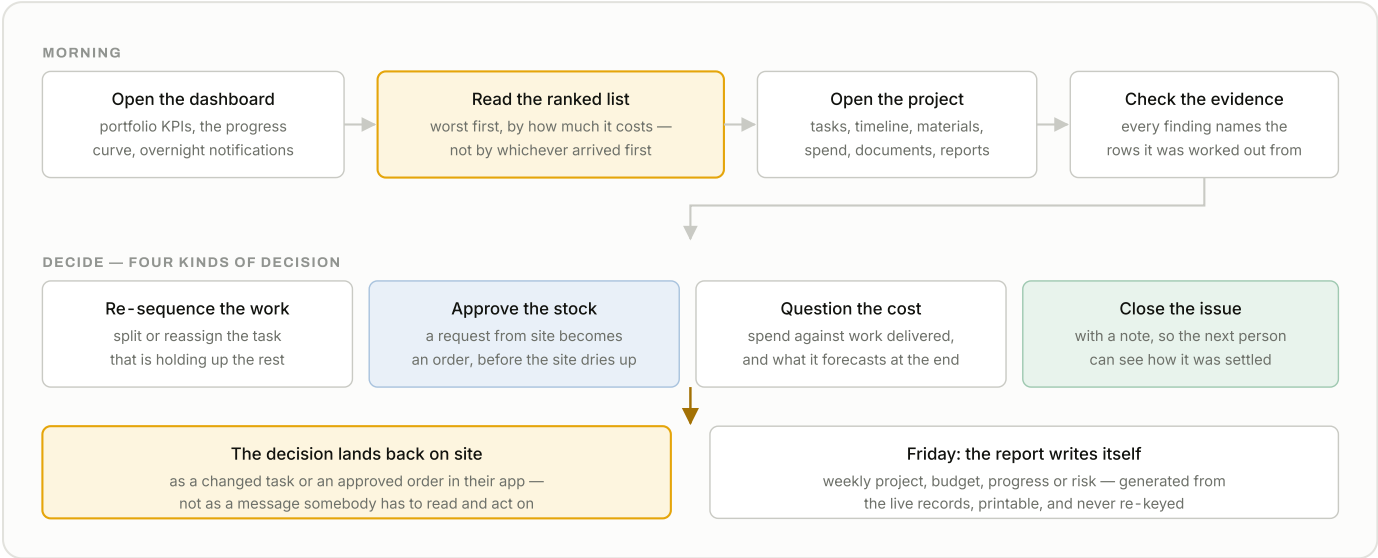


Figure 5 — The project manager's day. The morning row is reading, the middle row is deciding, and the bottom row is the decision arriving where the work is. The loop closes without anyone writing an email.

What the manager owns that nobody below them does: the budget, the programme, and the right to change either. What they do *not* own is access — they cannot create an account, change somebody's role, or remove a project from the portfolio. Running a site and controlling who can see it are different jobs, and the second belongs to the admin.

09 The admin — running the platform, not the build

A different job from the project manager's, and a deliberately separate console: most sessions never open it, and it should not weigh on the workspace everybody does.

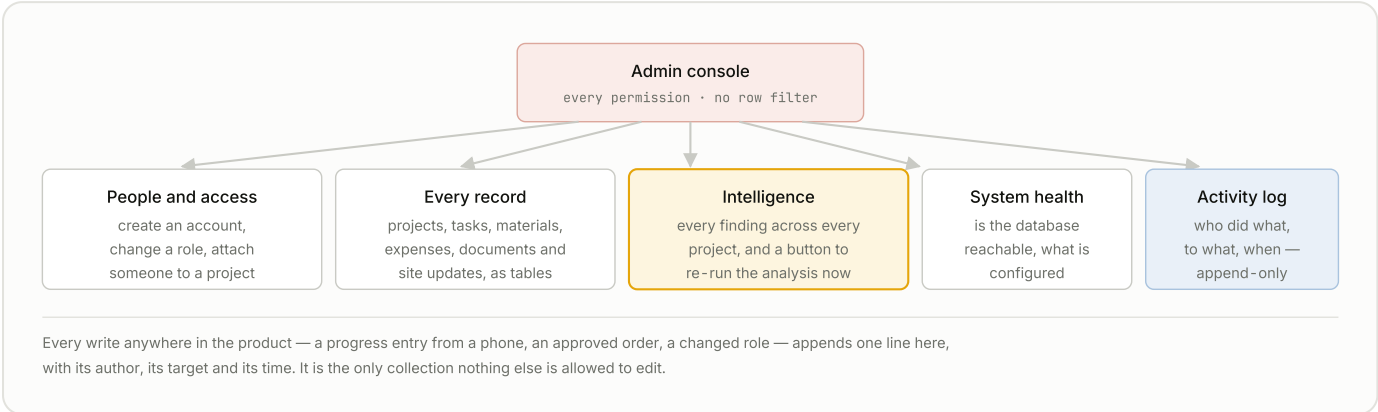


Figure 6 — What the admin sees. Four of the five panels are read-and-correct; the activity log is the one that is written to by everything and edited by nothing, which is what makes it usable as an audit trail.

Two protections are worth naming. Changing your own role or deleting your own account is refused, and the system will not let the last remaining admin be removed — a platform that can be locked out of by one careless click is not administered, it is abandoned. Reaching the admin workspace also requires the password again, so an unattended signed-in laptop does not hand over the organisation.

10 What happens when the daily report is submitted

The single most consequential action in the product. One tap at six in the evening, and seven things change.

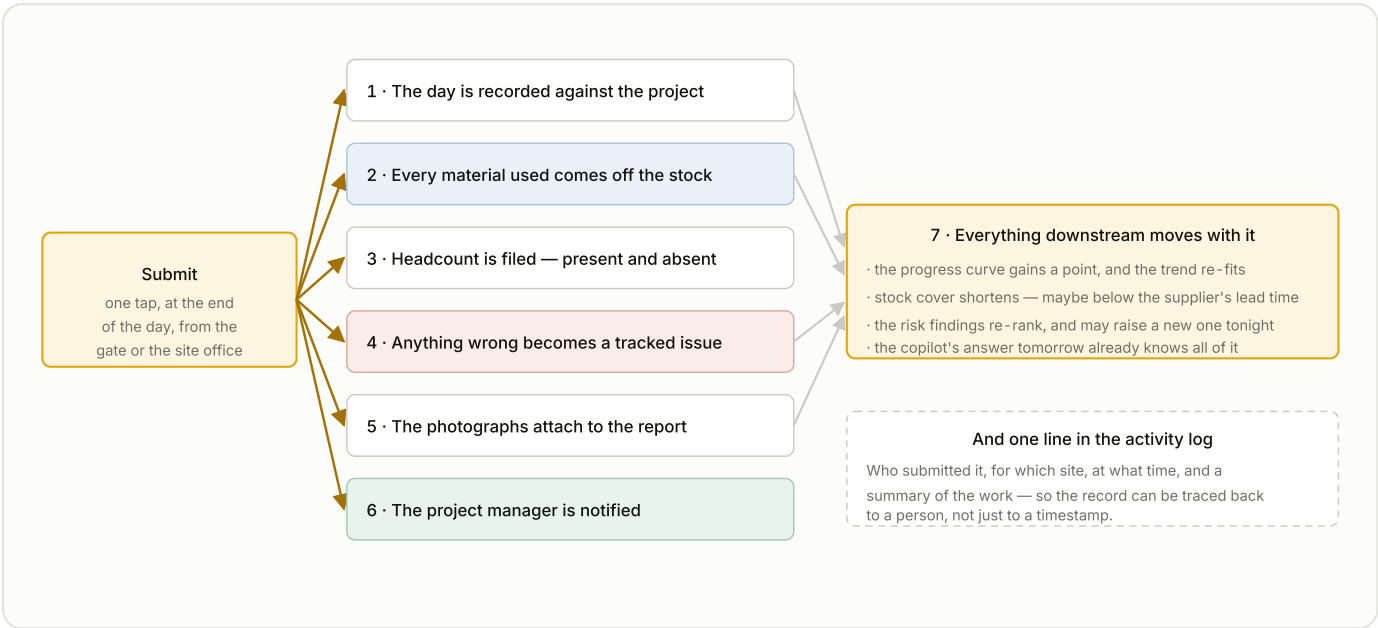


Figure 7 — One submission, seven consequences. The wizard collects work, headcount, consumption, issues and photographs together because that is how a day is remembered — splitting them into five forms would mean five chances to stop halfway. Note item 4: a problem typed into the report becomes a tracked issue with a severity, rather than a line of prose in a paragraph a manager may never scroll to.

11 What happens when a document is uploaded

Someone drops in a bill of quantities or a supplier's invoice. Four things happen to it before it is stored, and what comes out the other end is rows, not a file in a folder.

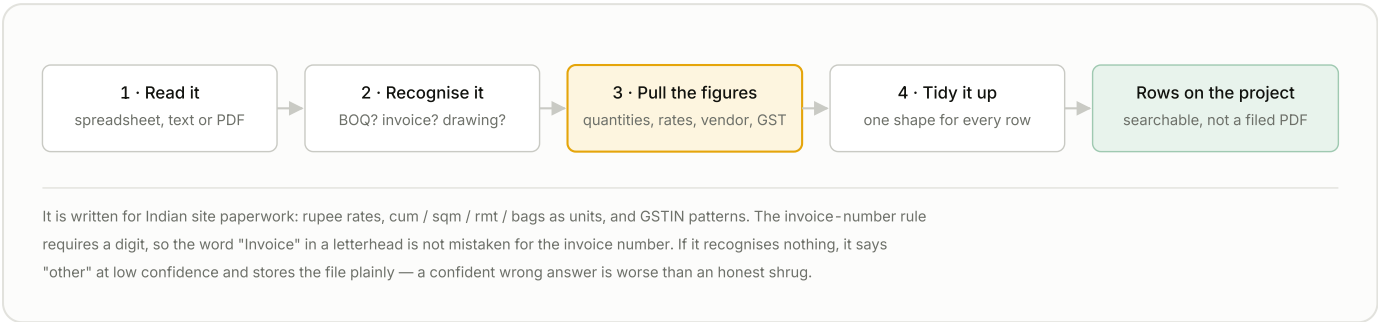


Figure 8 — Upload to rows. The confidence score travels with the extraction, so a manager can see whether a figure was read cleanly or guessed at from a messy scan.

12 What happens when the system finds a risk

Nobody asks it to. Three checks run over every project, and anything worth saying is raised as a finding — with the numbers behind it, a plain-language summary and one recommended action.

WHAT IT WATCHES	WHAT MAKES IT SPEAK UP	WHAT IT TELLS YOU TO DO
Schedule	Actual progress has fallen behind where the programme says it should be, judged against the rate the site has actually been building at over recent reports.	Re-sequence the critical path, or review the lagging tasks before the gap widens.
Budget	Spending has run ahead of the work it bought — and what that projects the final cost to be, against the budget.	Freeze non-critical orders and re-quote the largest open category.
Materials	Stock will run out before a replacement could arrive, judged against that supplier's lead time rather than a fixed number of days.	Raise the order by a named date — the last day it can still arrive in time.
Tasks	Several tasks are past their deadline, and the oldest is holding up everything sequenced after it.	Reassign or split the oldest one so the dependent phases can start.

Findings are ordered by how much they cost, not by when they appeared: a sixteen-point schedule slip is put above two overdue tasks. Each one carries a confidence score, and that score drops honestly when the evidence is thin — a project with fewer than three site reports gets a lower number, because a trend fitted to two points is not a trend.

13 What happens when someone asks BuildSync

"Which projects are delayed?" "Why is Skyline Tower behind?" "Which materials are running low?" The copilot answers from the user's own records, and shows which ones it used.

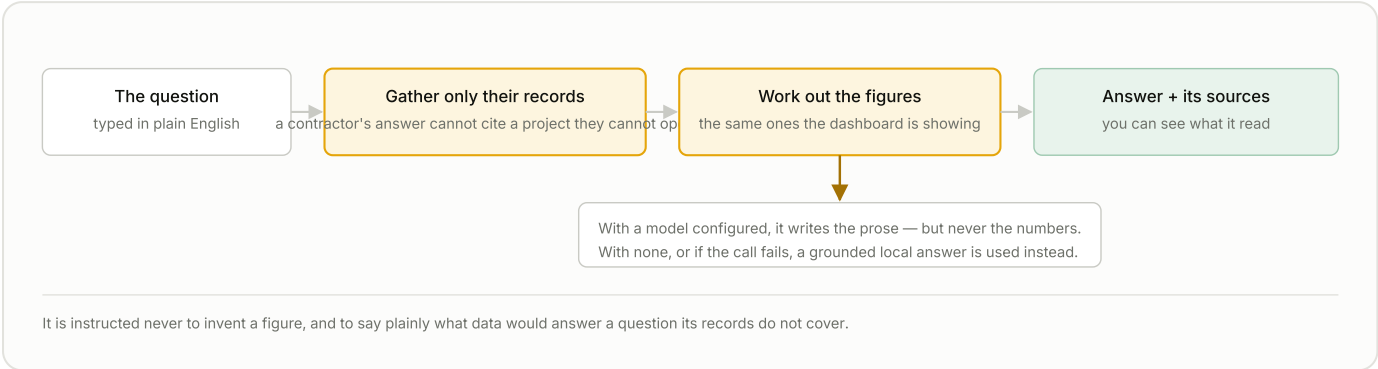


Figure 9 — Grounded, then written. The order matters: the figures are computed before any prose is generated, so the answer cannot drift from the dashboard. An outage costs style, not correctness.

14 Reports — who generates what

REPORT	WHO RUNS IT	THE QUESTION IT SETTLES
Daily site	Site manager	What happened on site today, with headcount and consumption
Weekly project	Project manager	Where the project stands, and what moved this week
Budget	Project manager	What has been spent, on what, against what was allowed
Progress	Project manager	Planned against actual over the period, as a curve
Risk	Manager or admin	What is most likely to go wrong next, ranked

Each is generated from live records at the moment it is run, not assembled by hand, and each is scoped to what the person running it may see — so a contractor's export cannot contain a project they cannot open.

15 What it does not do yet

Stated plainly, because a report that only lists what works is not a report.

Honest limits

- **The delay predictor is trained on generated data.** Alongside the three deterministic checks there is a trained model that scores how likely a project is to finish late. It performs well against the data it was built from, but no real project has yet been scored and its outcome watched, so it is presented as a second opinion and never as the only one. Where a project is missing a figure the model needs, it reports the project as unscored rather than filling the gap in — a missing percentage beside a real one is better than a made-up one.
- **Scanned drawings are not read.** The document pipeline pulls text out of files that contain text. A photographed or scanned drawing contains none, and needs character recognition that is not built.
- **Everything is local.** Nothing here is configured for deployment — no certificate handling, no secret manager, no way to revoke a token early, no rate limiting.
- **Notifications live in the product.** The manager is told inside BuildSync. Nothing is sent to a phone, an email or a messaging app yet, which is where a site manager actually is at six in the evening.

Next, in the order it is worth doing

1. **Push the notification out of the product** — to the phone. Every escalation in this report ends with "the manager is notified", and that is only true if they are looking.
2. **Record what actually happened.** Keep the forecast completion date beside the real one for every project, so the predictions can be measured rather than argued about.
3. **Read scanned drawings**, which is what most site drawings actually are.
4. **Harden it for a real site:** revocable sessions, rate limiting, uploads in object storage, secrets outside the filesystem.

The thesis, restated. Construction software usually fails not by computing the wrong number but by computing the same number three times and showing three answers. BuildSync has one real commitment: every figure is worked out in exactly one place, every finding carries the records it came from, and anything the data does not support is withheld rather than estimated. Everything in this report — the four workspaces, the one-tap day, the seven consequences of a single submission — follows from that.